개발 단계 보고서 · 10단계

MARTIALDEX — 카메라 기반 AI 자세 분석 시스템

작업 일자: 2026-08-14 · 작성: Claude (개발 어시스턴트)

1. 핵심 원칙

> 이 기능은 사용자의 실제 무술 실력을 완벽하게 판정하는 시스템이 아니다. 카메라에서 추출한 신체 관절 위치·움직임을 기반으로 "자세 기준 적합도"를 분석하는 참고용 시스템이며, 실제 훈련에서는 전문 지도자의 피드백이 필요하다는 안내를 UI 곳곳에 표시한다.

2. 사용한 무료 Pose 라이브러리

MediaPipe Tasks Vision(`@mediapipe/tasks-vision`, Google 무료 오픈소스) + `pose_landmarker_lite` 모델. nose·양쪽 shoulder·elbow·wrist·hip·knee· ankle 랜드마크를 사용한다. WASM 런타임과 모델 파일을 `public/mediapipe/`에 직접 번들링해 **완전 로컬 처리**(외부 서버 요청 0건, CSP `connect-src 'self'` 유지) — 유료 API 없음, 카메라 영상은 저장하지 않는다.

3. 지원 기술 목록 (16개)

무술	기술
복싱	Jab, Cross, Hook, Uppercut, High Guard
킥복싱	Jab, Cross, Round Kick, Front Kick
무에타이	Teep to Leg, Lead Round Kick / Rear Round Kick
태권도	앞차기, 돌려차기
레슬링	Wrestling Stance, Level Change, Penetration Step

실제 `data/processed/*.json`의 기술 ID만 참조했고(`scripts/validate-pose-config.mjs`로 검증), 나머지 1,237개 기술은 "카메라 자세 분석을 지원하지 않습니다"로 표시된다.

4. 분석 지표 (기술마다 다름, 순수 기하 계산)

`basicStance`(무릎 굽힘 각도) · `armPosition`(팔꿈치 각도/가드 손 위치) · `rotation` / `hipRotation`(어깨·골반 회전량) · `balance`(골반 중심 흔들림) · `headPosition`(머리 정렬) · `supportLegStability`(지지 발목 흔들림) · `kneePosition`(무릎 상승량) · `legMovement`(발 이동 거리) · `levelChange`(골반 하강량). 랜덤 점수는 전혀 없으며, 전부 캡처된 프레임의 실제 좌표에서 계산한다.

5. 점수 계산 방식

- 측정값과 기준값의 편차를 0~100으로 환산(`scoreFromDeviation`), 기준값/허용오차는 기술마다 `data/curated/poseAnalysis.json` 의 `params` 로 다르게 설정.
- 동작 구간(시작 → 동작 → 최대 동작 → 복귀)을 "가장 많이 뻗은/올라간" 프레임을 자동 탐색해 비교.
- 잘함(80+) / 보통(60~79) / 개선 필요(0~59) 경계값은 `SCORE_THRESHOLDS` 상수 하나로 관리.
- 핵심 랜드마크가 충분히 잡히지 않으면 점수를 억지로 만들지 않고 "포즈를 인식하지 못했습니다"를 반환.

합성 좌표 테스트로 직접 검증: 동일 입력 → 동일 출력(결정론적), "좋은 자세(81점) > 나쁜 자세(55점)" 방향성 확인 — 무작위 생성이 아님을 확인했다.

6. Gemini AI 연동

점수 계산은 로컬 알고리즘이 수행하고, Gemini는 이미 계산된 결과를 이해하기 쉽게 설명하는 역할만 한다. 시스템 프롬프트에 "이 점수는 코드가 계산했다, 너는 설명만 한다"와 "카메라로 확인 가능한 요소만 근사 측정한 것"이라는 경계를 명시했다.

7. 저장 방식

`martialdex:pose-records` (localStorage) — `{recordId, techniqueId, martialArtId, score, status, metricScores, analyzedAt}` . 기존 커리큘럼 기능과 동일한 repository → service → hook 계층 구조를 재사용했다. 카메라 영상 프레임은 분석 직후 즉시 버려지고 디스크에 절대 쓰이지 않는다 — 저장되는 것은 점수/세부 지표/날짜뿐이다. "나의 기술" 페이지에서 기술별/전체 삭제, JSON 내보내기(영상 미포함)를 제공하며, 삭제 전 항상 확인 메시지를 표시한다. 커리큘럼의 "학습 완료"와 자세 분석 점수는 완전히 분리된 데이터로 관리된다.

8. 새로 만든 / 수정한 파일

신규: `src/lib/poseAnalysis/` {types, landmarks, geometry, metrics, scoring, poseAnalysisConfig, analyzeTechnique, poseEstimator, recordsRepository, recordsService, usePoseRecords}.ts · `src/components/` {PoseAnalysisModal, PoseHistoryModal, ScoreHistoryChart}.tsx · `src/pages/MySkills.tsx` · `data/curated/poseAnalysis.json` + `POSE_ANALYSIS.md` · `scripts/validate-pose-config.mjs` · `public/mediapipe/` (WASM+모델) · `.devtools/qa-pose.mjs`
수정: `electron/main.cjs` (카메라 권한), `index.html` (CSP), `ai/context.ts` + `AiAskModal.tsx` , `TechniqueDetail.tsx` , `CurriculumDayDetail.tsx` , `Header.tsx` / `App.tsx` , `global.css` , `README.md` .

9. 테스트 결과

17 / 17	0
Playwright(fake camera) 통과	콘솔 에러

Playwright + Electron에서 `--use-fake-device-for-media-stream`으로 **실제 MediaPipe 파이프라인을 종단간 검증**: 권한 요청 → 카메라 스트림 → 실시간 포즈 오버레이 → 녹화 → 점수 산출 또는 "인식 실패" 안내까지 전부 확인. tsc/build 통과, 기존 AI(59/59)·커리큘럼(38/38) 회귀 스위트도 재실행해 전부 통과.

테스트 중 발견하고 수정한 실제 버그 3건

> ① `session.defaultSession`을 `app.whenReady()` 이전에 호출해 앱이 아예 실행되지 않던 문제 (모든 사용자에게 영향을 줬을 심각한 버그).

> ② 포즈가 인식되지 않으면 녹화 종료 타이머 체크 자체가 실행되지 않아 무한 대기 상태에 빠지는 문제 — 실사용에서 조명이 나쁘거나 사용자가 프레임 밖으로 나가면 발생 가능했음.

> ③ 새로 추가된 버튼들로 인해 `.detail-actions`가 모바일에서 가로 스크롤을 유발하던 CSS 회귀 (`flex-shrink:0`이 wrap을 무력화).

10. 현재 제한사항

- 카메라 1대 기준 — 몸이 겹치거나 카메라를 등지는 기술(클린치, 백 컨트롤 등)은 다루지 않음.
- 좌/우를 자동으로 판별("더 많이 움직인 쪽")하므로 아주 미세한 동작은 반대쪽이 선택될 수 있음.
- GPU 가속 기본값 — GPU가 없는 환경에서는 느려질 수 있음.

11. 1,253개 전체 기술로 확장하는 방법

1. 실제 기술 `id` 확인(새로 만들지 않음)
2. 기존 계산기(`METRIC_CALCULATORS`)로 충분한지 확인, 부족하면 `metrics.ts`에 추가
3. `data/curated/poseAnalysis.json`에 항목 추가
4. `node scripts/validate-pose-config.mjs`로 검증
5. 카메라 한 대로 분석하기 어려운 기술은 무리하게 추가하지 않는다